

Software Testing Interview Questions

Every interview question from the Test Engineer Beginners Roadmap - with quick answers, detailed answers and real project examples for every single one.

100

questions & answers

11

modules covered

3

levels: beginner to advanced

How to use this pack

- Start with the quick answer - that is your interview one-liner.
- Study the detailed answer to survive follow-up questions.
- Adapt the real project example into your own experience story.
- Practise out loud. Reading is not rehearsal.

Contents

Module 1 - Software Testing Fundamentals	1
10 questions	
Module 2 - Manual Testing	5
9 questions	
Module 3 - SQL & Database Testing	8
9 questions	
Module 4 - API Testing	11
9 questions	
Module 5 - Programming for Testers	15
9 questions	
Module 6 - Automation Testing	18
9 questions	
Module 7 - Mobile Testing	21
9 questions	
Module 8 - DevOps & CI/CD	24
9 questions	
Module 9 - Advanced Testing	27
9 questions	
Module 10 - AI for Software Testing	31
9 questions	
Module 11 - Career Preparation	35
9 questions	

MODULE 1

Software Testing Fundamentals

Q1 BEGINNER

What is software testing and why is it important?

QUICK ANSWER

Evaluating software to find defects and provide information about quality before users are affected.

DETAILED

Testing verifies the product against requirements and validates it against user needs. Its real value is information: it tells the team what risks exist so they can make informed release decisions. It is not only about finding bugs - it is about preventing defects, protecting users and reducing business risk.

REAL PROJECT EXAMPLE

On an e-commerce project, testing the checkout flow before a sale weekend caught a payment rounding bug that would have overcharged customers - a five-minute fix that avoided refunds, complaints and reputation damage.

Q2 BEGINNER

Explain severity vs priority with an example.

QUICK ANSWER

Severity = technical impact of the defect; priority = how urgently the business wants it fixed.

DETAILED

Severity is assessed by QA based on how badly the defect breaks the system; priority is set with the business based on urgency. They are independent: a crash in a rarely-used admin report is high severity, low priority; a typo in the company name on the homepage is low severity, high priority.

REAL PROJECT EXAMPLE

We once had a spelling mistake in the CEO's name on a press-release page - trivial technically, but it was fixed the same hour because of brand impact, while a crash in a legacy export waited for the next sprint.

Q3 BEGINNER

What is the difference between smoke and sanity testing?

QUICK ANSWER

Smoke: is the build stable enough to test at all? Sanity: does this specific fix or feature behave after a change?

DETAILED

Smoke testing is broad and shallow, run on every new build to check critical paths (launch, login, navigation). Sanity testing is narrow and deep, run after a change to verify the affected area works before wider testing. Smoke gates the build; sanity gates the change.

REAL PROJECT EXAMPLE

Our CI ran a 10-minute smoke suite on every build - if login failed, the build was rejected immediately. When a payment fix landed mid-sprint, we sanity-tested the payment flow first before committing to the full regression run.

Q4 INTERMEDIATE**How does the STLC fit inside an agile sprint?****QUICK ANSWER**

All STLC phases still happen - compressed and continuous within each sprint rather than as sequential gates.

DETAILED

In agile, requirement analysis happens during refinement, test planning at sprint planning, case design alongside development, execution as stories complete, and closure at the sprint review. Testing shifts left: testers question acceptance criteria before code exists, and automation runs continuously instead of in an end phase.

REAL PROJECT EXAMPLE

In my team, I wrote test scenarios from acceptance criteria during refinement and reviewed them with the developer before they started coding - half the potential bugs died as questions in that conversation instead of appearing as defects.

Q5 INTERMEDIATE**A developer says your bug is "not reproducible". What do you do?****QUICK ANSWER**

Re-verify with exact steps, environment, data and evidence, then pair with the developer to isolate the difference.

DETAILED

First re-reproduce it yourself and tighten the report: exact build, environment, test data, timing, and a screen recording. Check for environment or data differences between your setup and theirs. If it is intermittent, capture logs and note frequency. Then pair with the developer - most "not reproducible" bugs are environment or data mismatches.

REAL PROJECT EXAMPLE

A checkout bug only reproduced with a saved card from an older account schema. Adding the exact test account and a video to the report turned "cannot reproduce" into a fix within a day.

Q6 INTERMEDIATE**How do you decide which testing types to apply to a new feature?****QUICK ANSWER**

Start from risk: what could fail, who does it hurt, and how badly - then pick the types that address those risks.

DETAILED

Map the feature's risks: functional correctness (functional/integration tests), impact on existing behaviour (regression), data handling (database/API checks), user experience (usability/exploratory), and non-functional concerns (performance, security, accessibility) where the feature warrants it. Coverage follows risk, not a fixed checklist.

REAL PROJECT EXAMPLE

For a new file-upload feature we prioritised boundary tests (file size/type), security checks (malicious files), and integration with storage - but skipped performance testing because volumes were tiny. That reasoning was documented in the test plan.

Q7 ADVANCED**How do you decide what NOT to test when time is short?****QUICK ANSWER**

Risk-based testing: rank by probability x impact, cover the top risks deeply, consciously accept and communicate the rest.

DETAILED

I classify areas by user impact and change frequency. Recently changed, business-critical, or historically fragile areas get full attention; stable, low-traffic areas get smoke-level coverage or none. Crucially, what is NOT tested is stated explicitly in the test report so the release decision owns the residual risk knowingly.

REAL PROJECT EXAMPLE

Before a hard launch deadline we skipped deep testing of the admin reporting module (stable for 6 releases, internal users only) and spent that time on the redesigned checkout. The skip was documented and accepted by the product owner - and checkout shipped clean.

Q8 ADVANCED**What does "shift-left testing" mean in practice - beyond the buzzword?****QUICK ANSWER**

Moving quality activities earlier: testers shaping requirements, reviewing designs and writing tests before/with the code.

DETAILED

Practically it means testers join refinement to challenge acceptance criteria, static-test requirements and designs, define test cases before development (sometimes as executable specs), and wire automation into CI so feedback arrives in minutes. The goal is preventing defects - the cheapest bug is the one never written.

REAL PROJECT EXAMPLE

On one project we started reviewing API contracts before implementation. A missing error-code definition caught in review would previously have surfaced as three separate integration bugs across teams.

Q9 ADVANCED**How would you introduce a testing process to a team that has none?****QUICK ANSWER**

Start small and visible: risk-based smoke checklist, a defect workflow, then grow coverage where it earns trust.

DETAILED

I would assess the current pain (escaped defects, hotfix frequency), introduce a lightweight defect lifecycle and a smoke checklist for releases first - immediate visible value. Then add test scenarios for new stories, basic regression automation for critical paths, and metrics (escape rate, time-to-verify) to prove impact and justify further investment.

REAL PROJECT EXAMPLE

At a startup with zero QA, a one-page release checklist cut production hotfixes in half within a month - that credibility bought buy-in for proper test cases and a Playwright smoke suite in CI.

Q10 INTERMEDIATE**What is the difference between a test plan and a test strategy?****QUICK ANSWER**

Strategy = the organisation's high-level, stable approach to testing; plan = the project-specific document of scope, resources and schedule.

DETAILED

A test strategy defines long-lived principles: levels of testing, automation approach, environments, tools, risk appetite. A test plan applies them to one project/release: what is in and out of scope, who tests what, when, entry/exit criteria, and named risks. Strategy rarely changes; a plan changes with every release.

REAL PROJECT EXAMPLE

Our company strategy mandated "API-first automation, exploratory for new UI". My release plan for the payments feature translated that into 40 API tests, 2 exploratory charters, entry criteria (build passes smoke) and an exit report the PO signed off.

MODULE 2

Manual Testing

Q1 BEGINNER**Walk me through the fields of a good test case.****QUICK ANSWER**

ID, title, preconditions, numbered steps, test data, expected result per step, actual result, and status.

DETAILED

A test case needs a unique ID and a title that says what it verifies; preconditions describing required state; steps numbered so anyone can follow them; the exact test data; an expected result precise enough to judge pass/fail without interpretation; and fields for actual result and status filled during execution.

REAL PROJECT EXAMPLE

In my last project, our template also had a "requirement ID" column linking each case back to the acceptance criterion it covered - that traceability showed us instantly which requirements had no coverage.

Q2 BEGINNER**What is the bug lifecycle in your words?****QUICK ANSWER**

New -> Assigned -> In Progress -> Fixed -> Retest -> Closed, with Reopened and Deferred as the detours.

DETAILED

A tester logs a defect (New); a lead triages and assigns it; the developer fixes it and marks it Fixed; QA retests on the new build - pass means Closed, fail means Reopened. Bugs can also be Deferred (valid but postponed) or Rejected (not a bug/duplicate). The tester who raised it should always do the closing retest.

REAL PROJECT EXAMPLE

We added a rule that every Reopened bug needed a comment explaining what still failed - reopen rates dropped because developers started retesting their own fixes before handing them back.

Q3 BEGINNER**What is the difference between positive and negative testing?****QUICK ANSWER**

Positive proves valid input works; negative proves invalid input fails safely with clear feedback.

DETAILED

Positive scenarios follow the happy path with valid data - they prove the feature does what it should. Negative scenarios use invalid data, wrong order of actions, or abuse (empty fields, huge inputs, special characters) - they prove the system rejects them gracefully without crashes, data corruption or unclear errors. Most escaped production bugs live on the negative side.

REAL PROJECT EXAMPLE

For a money-transfer form I tested transfers of £0, negative amounts, more than the balance, and 0.001 pence - the negative amount case revealed you could actually credit your own account.

Q4 INTERMEDIATE**Acceptance criteria are vague. What do you do as the tester?****QUICK ANSWER**

Raise questions in refinement before development starts - turn ambiguity into testable, agreed statements.

DETAILED

I list concrete ambiguities as questions ("What happens at exactly 100 characters? Which currencies? What does 'fast' mean in numbers?") and push for the answers to be written back into the criteria. If the story is already in development, I agree the answers with the PO and developer in the ticket so the interpretation is documented, not assumed.

REAL PROJECT EXAMPLE

A story said "user can search products". My questions - partial matches? case sensitivity? empty results message? - became five extra acceptance criteria, and two of them changed what the developer had planned to build.

Q5 INTERMEDIATE**How do you prioritise what to test when a release is tomorrow?****QUICK ANSWER**

Risk first: new/changed features, business-critical flows, and historically fragile areas - in that order.

DETAILED

I split coverage into must-test (whatever changed this release, plus revenue-critical paths like login/checkout), should-test (integrations touching the changes), and accept-risk (stable, untouched areas - smoke level only). I communicate that split explicitly so the release decision includes what was NOT covered.

REAL PROJECT EXAMPLE

For a hotfix release we tested only the fixed flow, its module (sanity), and a 15-minute critical-path smoke. The email to stakeholders listed exactly that, so sign-off was informed, not assumed.

Q6 INTERMEDIATE**Severity and priority: who sets each, and can you give a high-priority/low-severity example?****QUICK ANSWER**

QA proposes severity from technical impact; business/lead sets priority from urgency. Homepage typo = low severity, high priority.

DETAILED

Severity is objective - how badly does it break the system (crash, data loss, cosmetic)? Priority is a business decision - how soon must it be fixed given impact on users, revenue and reputation. They often align but not always, and arguing them separately avoids fights: you can concede priority without misclassifying severity.

REAL PROJECT EXAMPLE

A legal-disclaimer line missing on one page was cosmetic severity-wise but compliance-critical - fixed same day. Meanwhile a reproducible crash in an internal admin tool used monthly waited two sprints.

Q7 ADVANCED**How would you measure and improve the quality of the team's bug reports?**

QUICK ANSWER

Track reopen rate, "needs info" bounce rate and time-to-fix; fix the gaps with a template and examples.

DETAILED

Poor reports show up as measurable friction: developers bouncing bugs back for information, high reopen rates, long triage. I'd introduce a lightweight template (title format, steps, expected/actual, environment, evidence), pair-review reports for a sprint, and show before/after metrics. The goal is reports that a developer can act on within minutes of opening.

REAL PROJECT EXAMPLE

After introducing a "one bug = one report, one video each" rule, our average time from bug-assigned to fix-started dropped from a day to a couple of hours - developers stopped context-hunting.

Q8 ADVANCED**A stakeholder asks "why did QA miss this production bug?" How do you respond?**

QUICK ANSWER

Blameless root-cause: was it untested, untestable, out of scope, or a coverage gap - then fix the process, not the person.

DETAILED

I trace the escape honestly: was the scenario in our cases (execution gap), missing from them (design gap), impossible in our environments (infrastructure gap), or an accepted risk (communication gap)? Each root cause has a different fix - new regression case, better data, environment parity, or clearer risk sign-off. I present it as "here's what we're changing", not excuses.

REAL PROJECT EXAMPLE

A production-only payment bug came from a config we couldn't replicate. The outcome wasn't "test harder" - it was getting a staging environment with production-parity payment config, which caught two similar issues the next quarter.

Q9 INTERMEDIATE**What is a Requirement Traceability Matrix (RTM) and why does it matter?**

QUICK ANSWER

A mapping of requirements to test cases (and defects) proving every requirement has coverage - and showing the impact of changes.

DETAILED

The RTM links each requirement id to the test cases that verify it and the defects raised against it. It answers two critical questions instantly: "is anything untested?" (a requirement with no linked cases) and "what must we retest if this requirement changes?". In tools like Jira/Zephyr it is generated from links rather than maintained by hand.

REAL PROJECT EXAMPLE

A client audit asked us to prove coverage of 200 regulatory requirements. The RTM answered in minutes - and exposed 7 requirements with zero tests, which became that sprint's priority.

MODULE 3

SQL & Database Testing

Q1 BEGINNER

Why does a tester need SQL at all?

QUICK ANSWER

To verify the truth behind the UI: what was actually saved, changed or deleted in the database.

DETAILED

The UI shows what the application WANTS you to see; the database shows what actually happened. SQL lets testers verify saves, check calculations, prepare test data, investigate bugs (is it a display issue or a data issue?) and validate migrations. It also halves debugging time because you can tell developers WHERE the problem is.

REAL PROJECT EXAMPLE

A "wrong balance" bug looked like a calculation error, but a quick `SELECT` showed the stored balance was correct - the bug was in the display formatting layer. That one query redirected the fix to the right team immediately.

Q2 BEGINNER

Explain the difference between WHERE and HAVING.

QUICK ANSWER

WHERE filters individual rows before grouping; HAVING filters the groups after aggregation.

DETAILED

WHERE runs first, on raw rows - it cannot see aggregate values. GROUP BY then buckets the surviving rows, and HAVING filters those buckets using aggregates like COUNT or SUM. Rule of thumb: filtering on a column value -> WHERE; filtering on an aggregate -> HAVING.

REAL PROJECT EXAMPLE

To find customers with more than 3 failed payments this month: `WHERE status = 'failed' AND month = this month, GROUP BY customer, HAVING COUNT(*) > 3` - both filters, different stages.

Q3 BEGINNER

What is a JOIN and when did you last use one in testing?

QUICK ANSWER

A JOIN combines rows from related tables via matching keys - essential because real data is spread across tables.

DETAILED

Databases are normalised: orders reference customers by id, order_items reference orders. To verify anything meaningful you join them back together. INNER JOIN gives only matches; LEFT JOIN keeps everything from the first table - which is how you find records MISSING a relationship (the join column is NULL).

REAL PROJECT EXAMPLE

To verify an order-confirmation email job, I joined orders LEFT JOIN emails_sent and looked for NULL email rows - instantly listing every order whose confirmation was never queued.

Q4 INTERMEDIATE**How would you test a data migration?****QUICK ANSWER**

Count checks, sample deep-comparison, constraint validation, and business-rule spot checks - before and after.

DETAILED

First reconcile row counts per table and per meaningful segment. Then deep-compare a sample of records field by field (including edge cases: nulls, special characters, extremes). Verify constraints and relationships survived (no orphans, uniques intact). Finally run business checks - totals, balances, statuses - and test the application against the migrated data, not just the data itself.

REAL PROJECT EXAMPLE

In a CRM migration we reconciled counts by country and status, then diff-checked 500 sampled customers programmatically. The sample caught a truncation bug in address lines longer than 100 characters that counts alone would never show.

Q5 INTERMEDIATE**The UI shows 12 products but you expect 14. How do you investigate with SQL?****QUICK ANSWER**

Query the source table with the same filters the page should apply, then diff the ids against what renders.

DETAILED

I'd write the query that the page logically represents (active products, in stock, this category), compare the count, then SELECT the ids and diff against the UI. The two missing rows usually reveal the bug: a status flag, a null field the UI filter chokes on, or a pagination/limit issue. Now the bug report says WHY two products are missing, not just that they are.

REAL PROJECT EXAMPLE

Two "missing" hotels on a listings page turned out to have NULL star-ratings, and the UI filter silently excluded NULLs. The SQL diff made it a five-minute diagnosis and a precise bug report.

Q6 INTERMEDIATE**How do you test a stored procedure?****QUICK ANSWER**

Treat it like an API: known inputs -> verify outputs AND side effects on tables, including edge and error cases.

DETAILED

I identify its contract (parameters, return values, tables it modifies), prepare controlled input data, execute with typical, boundary and invalid parameters, then verify both the returned results and every table it should (and should not) have touched. Transactions matter: does it roll back cleanly on failure, or leave partial data?

REAL PROJECT EXAMPLE

Testing a monthly-interest procedure, the happy path was fine - but calling it twice for the same month double-credited interest. The missing idempotency check became the highest-severity finding of the release.

Q7 ADVANCED**What is data integrity and how do you test it beyond constraints?**

QUICK ANSWER

Constraints are the floor; real integrity is business truth - totals reconcile, states are consistent, nothing orphans.

DETAILED

Schema constraints (FKs, uniques, NOT NULL) catch structural corruption, but business integrity needs targeted queries: do order totals equal the sum of their items? Can a "shipped" order have no shipment record? Are there payments without orders? I keep a suite of such reconciliation queries and run them after big features, migrations and in production monitoring.

REAL PROJECT EXAMPLE

A nightly reconciliation query - bookings vs payments vs invoices - caught a race condition that created paid bookings with no invoice about once a week. No constraint could have expressed that rule.

Q8 ADVANCED**A report query is correct but takes 4 minutes. What do you look at as a tester?**

QUICK ANSWER

The execution plan: missing indexes, full table scans, unnecessary joins - and whether test data volume matches production.

DETAILED

Correctness is only half of database testing. I'd run EXPLAIN to see the plan, look for full scans on large tables and joins without indexed keys, and check whether the query fetches far more than it needs (SELECT *, no LIMIT). Critically, I'd test with production-scale data volumes - a query that flies on 1k test rows can crawl on 10M.

REAL PROJECT EXAMPLE

Our staging DB had 2% of production volume, so a dashboard "passed" testing then timed out on launch day. We started refreshing staging with anonymised production-scale data, and volume-related regressions stopped escaping.

Q9 INTERMEDIATE**Write a query to find duplicate records in a table.**

QUICK ANSWER

GROUP BY the columns that define a duplicate and keep groups HAVING COUNT(*) > 1.

DETAILED

SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1 lists duplicated emails. To see the full rows, join that result back to the table or use a window function: COUNT(*) OVER (PARTITION BY email) > 1. As a tester this finds data-integrity bugs - duplicates usually mean a missing unique constraint or a race condition in the code.

REAL PROJECT EXAMPLE

After a signup-flow change I ran the duplicate-email query on staging and found 14 doubles created by double-clicking the submit button - a missing unique constraint AND a missing UI debounce, two bugs from one query.

MODULE 4

API Testing

Q1 BEGINNER

What is an API and why do we test at the API level?

QUICK ANSWER

A contract for systems to exchange data without a UI - testing it is faster, more stable and catches logic bugs earlier than UI tests.

DETAILED

An API defines requests and responses between systems. Testing at this layer verifies business logic directly: no rendering, no flaky selectors, milliseconds per check. It also enables testing before the UI exists, precise error-case coverage (the UI often can't even trigger a 500), and clean integration checks between services.

REAL PROJECT EXAMPLE

Our checkout UI test took 40 seconds; the equivalent API test took 300ms. We covered 30 error combinations at API level - the UI could realistically trigger only three of them.

Q2 BEGINNER

Explain the common HTTP status code groups.

QUICK ANSWER

2xx success, 3xx redirection, 4xx client errors (your request is wrong), 5xx server errors (their code broke).

DETAILED

Key ones for testers: 200 OK, 201 Created, 204 No Content; 301/302 redirects; 400 Bad Request (malformed input), 401 Unauthorized (no/invalid credentials), 403 Forbidden (valid identity, no permission), 404 Not Found, 409 Conflict, 422 Unprocessable Entity, 429 Too Many Requests; 500 Internal Error, 502/503 gateway/availability. The 401 vs 403 and 400 vs 422 distinctions come up constantly.

REAL PROJECT EXAMPLE

A "login broken" incident was actually the API correctly returning 429 rate-limits under a bot attack - recognising the code family instantly reframed the bug from functional to security.

Q3 BEGINNER

What would you check in an API response beyond the status code?

QUICK ANSWER

Response body structure and values, headers, response time, and side effects in the database.

DETAILED

A 200 with wrong data is still a failure. I assert the body matches the contract (fields, types, values), check headers (content-type, caching, security headers), watch response time against SLA, and verify side effects - did the POST actually create the row, fire the event, send the email? Negative tests also verify the error body is useful, not just the code.

REAL PROJECT EXAMPLE

One endpoint returned 200 with an empty body when a downstream service failed - consumers treated it as success and lost data. We added assertions on required fields, not just status.

Q4 INTERMEDIATE**How do you test authentication and authorisation on an API?****QUICK ANSWER**

Auth: valid, invalid, expired, missing tokens. Authz: role x resource matrix - especially accessing OTHER users' data.

DETAILED

Authentication tests: no token (expect 401), malformed token, expired token, revoked token, token refresh flow. Authorisation tests: build a matrix of roles vs operations - can a regular user hit admin endpoints? Can user A read/update/delete user B's resources by swapping ids (IDOR)? Those object-level checks find the highest-impact security bugs testers can catch.

REAL PROJECT EXAMPLE

By simply changing /orders/1041 to /orders/1042 with a different user's token, I could read another customer's order including their address - a one-request test that became a critical security fix.

Q5 INTERMEDIATE**Your API test fails intermittently. How do you stabilise it?****QUICK ANSWER**

Find the nondeterminism: shared test data, ordering assumptions, timing/async, environment - then isolate and control it.

DETAILED

Common causes: tests sharing mutable data (fix with per-test data or setup/teardown), asserting on array order the API doesn't guarantee (assert on content, not position), async processing not yet complete (poll with timeout instead of fixed sleeps), and environment instability (retry policy + monitoring, but never blind retries that mask real bugs).

REAL PROJECT EXAMPLE

A flaky "get orders" test assumed newest-first ordering; the API guaranteed no order at all. Asserting "contains order X" instead of "orders[0] is X" fixed a month of noise.

Q6 INTERMEDIATE**How do you organise Postman for a real project rather than ad-hoc requests?****QUICK ANSWER**

Collections per service, folders per resource, environments per target, variables for everything repeated, tests on every request.

DETAILED

Structure: one collection per API/service, folders mirroring resources (auth, users, orders). All URLs use {{baseUrl}}; tokens are set by a login request into environment variables and referenced everywhere. Each request carries assertions (status, schema, key values). Collection runner + Newman turn the same collection into a CI-runable suite.

REAL PROJECT EXAMPLE

Our regression collection ran in CI via Newman on every backend merge - 180 requests in 90 seconds, failing the pipeline on any contract break before the frontend team ever saw it.

Q7 ADVANCED**What is contract testing and when would you need it beyond normal API tests?**

QUICK ANSWER

Verifying provider and consumer agree on the API shape - catching breaking changes between teams before integration.

DETAILED

Normal API tests check a running service. Contract tests capture what consumers actually rely on (fields, types, status codes) and verify the provider still satisfies it on every change - catching silent breaking changes (removed field, changed type) even when the provider's own tests pass. Valuable the moment separate teams/services deploy independently.

REAL PROJECT EXAMPLE

The backend renamed "postcode" to "postal_code" and their tests passed. A consumer-driven contract test failed their pipeline immediately - before three consumer apps broke in staging.

Q8 ADVANCED**How would you API-test an operation that is asynchronous (e.g. report generation)?**

QUICK ANSWER

Assert the accepted response (202 + job id), poll the status endpoint with a timeout, then verify the final artefact and failure paths.

DETAILED

Async flows split into: submission (202 Accepted, job id returned, validation errors rejected synchronously), progress (status endpoint transitions pending -> processing -> done, sensible timeout with clear failure), and result (artefact correct and retrievable, errors reported with reasons). Also test duplicate submissions, cancellation, and what happens when the worker dies mid-job.

REAL PROJECT EXAMPLE

Testing an export API, everything passed until we killed the worker mid-job: the status stayed "processing" forever. The team added a stuck-job watchdog and a "failed" state - found only because we tested the async lifecycle, not just the happy path.

Q9 INTERMEDIATE**What is API versioning and what should a tester check when a new version ships?**

QUICK ANSWER

Running v1 and v2 side by side so consumers migrate safely - test both versions' contracts, defaults and deprecation behaviour.

DETAILED

Versioning (URL /v2/, header-based, or content negotiation) lets providers evolve without breaking consumers. Testers verify: v2 delivers the new contract; v1 STILL behaves exactly as before (the forgotten half!); differences are documented; deprecation warnings/sunset headers appear; and error behaviour for unsupported versions is clear, not a bare 500.

REAL PROJECT EXAMPLE

When our team shipped /v2/orders, my regression suite kept running against /v1 too - it caught a shared

serialiser change that silently renamed a v1 field. v2 was perfect; v1 consumers would have broken in production.

MODULE 5

Programming for Testers

Q1 BEGINNER

How much programming does a tester really need?

QUICK ANSWER

Enough to read code, write clean test scripts, and debug failures: variables, conditions, loops, functions, and basic OOP.

DETAILED

You don't need to build products - you need to automate checks and understand what you're testing. That means comfortable fluency with data types, control flow, functions and collections; enough OOP to work in page-object frameworks; and debugging skill to diagnose failures. Depth grows naturally with the automation you build.

REAL PROJECT EXAMPLE

My first automation ticket only needed a loop and an assertion - but debugging why it failed on CI taught me more about scope and async than any course.

Q2 BEGINNER

What is the difference between a parameter and an argument?

QUICK ANSWER

Parameters are the names in the function definition; arguments are the actual values you pass when calling it.

DETAILED

function login(username, password) declares two parameters. Calling login("hari", "secret123") passes two arguments. In testing this matters because parameterised helpers are how one function serves many test cases - the essence of reusable test code.

REAL PROJECT EXAMPLE

Our createUser(role) helper had one parameter and was called with "admin", "editor" and "viewer" arguments across 40 tests - one function, every permission scenario.

Q3 BEGINNER

Explain variable scope and a bug it can cause in tests.

QUICK ANSWER

Scope is where a variable is visible. Shared/global state between tests causes order-dependent, flaky failures.

DETAILED

Variables declared inside a function/block exist only there; globals are visible everywhere - and mutable globals are poison in test suites: one test changes shared state, another silently depends on it, and the suite passes alone but fails together (or vice versa). Keep state local; create fresh data per test.

REAL PROJECT EXAMPLE

A suite stored the logged-in user in a module-level variable. Tests passed individually but failed in parallel - two tests fought over the same variable. Moving it into per-test fixtures fixed a month of "random" flakes.

Q4 INTERMEDIATE**Why is OOP useful in test automation specifically?****QUICK ANSWER**

It maps perfectly to frameworks: pages become classes (encapsulation), common behaviour is shared (inheritance), tests read like actions.

DETAILED

Encapsulation hides selectors inside page classes so UI changes are one-file fixes. Inheritance shares plumbing (navigation, waits) via base classes. Polymorphism lets different page types answer the same call (e.g. `isLoading()`) their own way. The result is tests that read as behaviour - `loginPage.login(user)` - not as DOM manipulation.

REAL PROJECT EXAMPLE

When our app rebranded and half the selectors changed, we fixed 9 page classes and zero of the 200 tests - that's encapsulation paying rent.

Q5 INTERMEDIATE**How do you handle errors in automation code - what should and shouldn't be caught?****QUICK ANSWER**

Catch expected, recoverable situations; never swallow assertion failures or unknown errors - let tests fail loudly with context.

DETAILED

`try/catch` belongs around genuinely optional operations (dismissing a cookie banner that may not appear) and cleanup (always release data/close sessions in finally). It must NOT wrap assertions or broad test bodies - catching everything converts real failures into silent passes. When re-throwing, add context: which page, which data, screenshot path.

REAL PROJECT EXAMPLE

A suite "went green" for two weeks while checkout was broken - someone had wrapped the test body in `try/catch` with an empty catch. We banned empty catches in code review after that.

Q6 INTERMEDIATE**Describe your debugging process for a failing automated test.****QUICK ANSWER**

Read the error fully, reproduce locally, isolate (test vs app vs environment), inspect state at the failure point, then fix the root cause.

DETAILED

First read the actual message and stack trace - half the answers live there. Reproduce: locally, same data, same browser. Classify: application bug, test bug (bad locator, timing assumption), or environment (data, config). Use breakpoints/pause modes and traces to inspect the real state at failure. Fix the cause, not the symptom - a `sleep()` that "fixes" it usually hides a race.

REAL PROJECT EXAMPLE

A test failed only on CI at 2am. The trace showed a date boundary: the test created "today" data that the assertion read as "yesterday" across midnight UTC. The fix was controlling the clock, not retrying the test.

Q7 ADVANCED**When would you choose composition over inheritance in a test framework?**

QUICK ANSWER

When behaviour is a capability, not an identity - inject helpers (API client, DB checker) rather than growing god base classes.

DETAILED

Deep inheritance chains (BasePage -> AuthPage -> FormPage -> ...) become rigid and force unrelated pages to carry unneeded behaviour. Composition - pages receiving the helpers they need (apiClient, dataFactory) - keeps units small, testable and mixable. Rule of thumb: inherit for is-a (LoginPage is a Page); compose for has-a/uses-a (a test uses an API client).

REAL PROJECT EXAMPLE

We replaced a 5-level base-class chain with a fixtures object injecting api, db and ui helpers per test - new engineers stopped asking "which base class do I extend?" and framework onboarding halved.

Q8 ADVANCED**How do you keep test code quality as high as production code?**

QUICK ANSWER

Same rules: reviews, linting, DRY helpers, meaningful names, no dead code - because flaky test code costs more than most prod code.

DETAILED

Test code IS production code for the QA team: it runs thousands of times, and its failures block releases. That means pull-request reviews for tests, linters/formatters in CI, shared helpers instead of copy-paste, clear naming (test names describing behaviour), and periodic refactoring sprints to kill duplication and flaky patterns before they metastasise.

REAL PROJECT EXAMPLE

We ran a "test debt" audit each quarter: delete dead tests, merge duplicates, fix the top 5 flaky ones. Suite runtime dropped 30% and trust in red builds returned - people stopped hitting re-run reflexively.

Q9 BEGINNER**How do you approach a live coding task in a QA interview?**

QUICK ANSWER

Clarify, think aloud, start simple, test your own code with edge cases - the process matters as much as the solution.

DETAILED

Restate the problem and ask about edge cases (empty input? nulls? case sensitivity?). Narrate your plan before typing. Write the simple working version first, then improve. Crucially for QA roles: test your own solution out loud with normal, boundary and invalid inputs - interviewers hire testers who instinctively attack their own code.

REAL PROJECT EXAMPLE

Asked to "find duplicates in an array", I finished with: "let me test it - empty array, single item, all duplicates, mixed types...". The interviewer later said that final minute was why I got the offer: I tested like a tester, not a developer.

MODULE 6

Automation Testing

Q1 BEGINNER

What should and shouldn't be automated?

QUICK ANSWER

Automate: stable, repetitive, high-value, objective checks. Don't: exploratory, one-offs, visual judgement, churning UIs.

DETAILED

Great candidates: smoke/regression suites, data-driven validations, API contracts, anything run per-release. Poor candidates: features still changing daily (maintenance burns the ROI), subjective assessments (does this feel right?), and true exploratory testing - automation repeats known checks; humans discover unknown problems. The equation is value x frequency vs build + maintenance cost.

REAL PROJECT EXAMPLE

We declined to automate a promo-banner feature that marketing redesigned weekly - manual 5-minute checks beat rewriting selectors every sprint. The stable checkout flow got the automation budget instead.

Q2 BEGINNER

Why does Playwright rarely need sleep() calls?

QUICK ANSWER

Auto-waiting: actions wait for elements to be actionable, and web-first assertions retry until they pass or time out.

DETAILED

Before clicking, Playwright waits for the element to be visible, stable, enabled and unobscured. Assertions like toBeVisible/toHaveText poll continuously up to the timeout. This kills the #1 flakiness source - hard-coded sleeps that are either too short (flaky) or too long (slow). A sleep in a Playwright test is almost always a smell.

REAL PROJECT EXAMPLE

Migrating a Selenium suite full of Thread.sleep(3000) to Playwright cut runtime 60% AND flake rate - the waits became exactly as long as needed, no more.

Q3 BEGINNER

Explain your locator strategy priority.

QUICK ANSWER

User-facing first: role + name, then label/placeholder/text, then data-testid; CSS/XPath structure only as a last resort.

DETAILED

Locators tied to what users perceive (getByRole("button", {name: "Pay"})) survive refactors and double as accessibility checks. data-testid is the stable engineering contract when semantics aren't enough. Structural CSS chains and positional XPath break on every layout tweak - they are the technical debt of test suites.

REAL PROJECT EXAMPLE

A rebrand changed every class name in the app; our role/testid-based suite needed 4 locator fixes. The legacy XPath suite next door needed 200.

Q4 INTERMEDIATE**Describe the Page Object Model and one thing people get wrong with it.****QUICK ANSWER**

One class per page exposing user actions and hiding locators. Common mistake: putting assertions inside page objects.

DETAILED

Page objects encapsulate locators + interactions (`loginPage.login(user)`); tests hold the assertions and business flow. Anti-patterns: assertion-stuffed page objects (couples checks to pages, hides intent), god objects covering whole apps, and leaking locators into tests. Keep pages dumb and actions readable - the test should read like a user story.

REAL PROJECT EXAMPLE

We refactored assertions out of page objects into tests; the same page methods then served smoke, regression and a11y suites - each applying different checks to identical actions.

Q5 INTERMEDIATE**How do fixtures improve on beforeEach/afterEach setup?****QUICK ANSWER**

Fixtures are composable, on-demand and self-cleaning - each test declares what it needs and gets exactly that.

DETAILED

`beforeEach` runs everything for every test whether needed or not, and cleanup logic drifts. Fixtures (Playwright's model) define a resource once - `loggedInPage`, `testUser`, `apiClient` - with setup AND teardown together; tests receive only the fixtures they request, enabling precise parallel-safe isolation and reuse across files.

REAL PROJECT EXAMPLE

Our "premiumUser" fixture created a user via API, logged in, and deleted the user afterwards. Forty tests used it in one line each; teardown lived in exactly one place.

Q6 INTERMEDIATE**Your suite is green locally but flaky in CI. Where do you look?****QUICK ANSWER**

Differences: parallelism and shared data, resources/headless timing, environment config, network - plus traces on retry.

DETAILED

CI differs in: parallel workers colliding on shared test data (isolate per worker), slower/headless machines exposing race conditions your fast laptop hides, missing env vars/config, and third-party dependencies behaving differently. Playwright traces + videos on retry usually show the truth. Fix causes; adding retries without investigation just buries real bugs.

REAL PROJECT EXAMPLE

Two workers both used "test@example.com". Locally (one worker) it passed; CI's four workers fought over the account. Worker-indexed users ended the mystery flakes overnight.

Q7 ADVANCED**How would you design an automation framework for a new project from scratch?**

QUICK ANSWER

Start thin: runner + POM + fixtures + data strategy + CI from day one; grow layers (API, reporting, parallel) as value proves.

DETAILED

Decisions in order: language matching the team, runner (Playwright Test brings runner/parallel/trace built in), structure (pages/, tests/, fixtures/, data/), naming and tagging conventions (@smoke/@regression), test-data strategy (API-seeded, isolated per test), CI wiring on day one - even 5 tests should run per PR - and reporting (HTML + traces). Resist building a "platform" before ten solid tests exist.

REAL PROJECT EXAMPLE

For a greenfield SaaS project we had 8 smoke tests in CI by the end of week one; the framework grew API helpers and visual checks only when suites demanded them. Eighteen months later it was 600 tests and still comprehensible.

Q8 ADVANCED**Playwright vs Selenium vs Cypress - how do you choose?**

QUICK ANSWER

By constraints: language/legacy -> Selenium; JS-only team & component focus -> Cypress; most new web projects -> Playwright.

DETAILED

Selenium: widest language/browser/grid ecosystem, integrates with everything, but more boilerplate and timing management. Cypress: superb DX inside its browser sandbox, JS/TS only, limited multi-tab/origin/browser control. Playwright: modern API, auto-wait, real multi-browser (incl. WebKit), parallelism, traces - the strongest default for new web automation. The right answer names trade-offs, not a religion.

REAL PROJECT EXAMPLE

We kept a mature Selenium suite (it worked; rewriting was negative ROI) but chose Playwright for the new customer portal - WebKit coverage and trace-based debugging sealed it.

Q9 INTERMEDIATE**Explain the test pyramid - and when you would deviate from it.**

QUICK ANSWER

Many fast unit tests, fewer integration/API tests, few E2E UI tests. Deviate when risk lives elsewhere - but always know why.

DETAILED

The pyramid optimises feedback speed and stability: unit tests are cheap and precise, API tests verify behaviour without UI flakiness, E2E covers only critical journeys. Inverting it (many E2E, few unit) gives slow, fragile suites. Legit deviations: heavy-UI products (visual/E2E matter more), legacy systems with no unit-test seam (API-heavy "diamond"), or compliance flows that must be tested end-to-end.

REAL PROJECT EXAMPLE

On a legacy app with zero unit tests, we built a diamond instead: 300 API tests as the backbone, 20 E2E journeys, and pushed for unit tests only in new code. Dogmatic pyramid-chasing would have stalled us for a year.

MODULE 7

Mobile Testing

Q1 BEGINNER

How is mobile testing different from web testing?

QUICK ANSWER

Device fragmentation, interruptions, gestures, sensors, networks, app lifecycle and store rules - the environment is hostile and varied.

DETAILED

Beyond a browser: thousands of device/OS combinations; system interruptions (calls, notifications); touch gestures; hardware (camera, GPS, biometrics); unreliable networks and offline modes; app lifecycle states (background/kill/resume); installs, updates and permissions; plus platform store requirements. Test strategy must sample this matrix by risk, not chase all of it.

REAL PROJECT EXAMPLE

Our web app "worked" as a mobile web view until we tested app-switching: returning from the camera permission dialog lost the entire form. Purely a mobile-lifecycle bug - invisible in any browser.

Q2 BEGINNER

When do you use emulators vs real devices?

QUICK ANSWER

Emulators for fast functional feedback and early CI; real devices for performance, sensors, networks and release sign-off.

DETAILED

Emulators/simulators are free, scriptable and fast to reset - ideal for development-loop functional tests. But they fake hardware: performance is misleading, sensors are simulated, and real-world networks/battery/thermal behaviour is absent. A sensible pyramid: broad emulator coverage continuously, a curated real-device set (top market devices + oldest supported) for critical flows and releases.

REAL PROJECT EXAMPLE

Animations were silky on the emulator and janky on a mid-range Samsung with 2GB RAM - the phone most of our users actually owned. That device joined the release checklist permanently.

Q3 BEGINNER

What would your smoke test for a mobile app include?

QUICK ANSWER

Install/launch, login, the top 3 user journeys, backgrounding/resume, and basic rotation/permissions.

DETAILED

Fresh install and launch (plus upgrade-in-place from the previous version), sign-in, the flows that matter commercially (browse, add, pay), background/foreground and kill/relaunch state, a rotation pass on key screens, and permission prompts behave sanely. Fifteen minutes, per build, ideally automated per merge.

REAL PROJECT EXAMPLE

Our 12-flow Maestro smoke ran on every PR in ~8 minutes on an emulator farm; the same flows ran nightly on 6 real devices. Upgrade-install testing caught a migration crash that fresh installs never showed.

Q4 INTERMEDIATE**Explain how an Appium test actually reaches the app.****QUICK ANSWER**

Your test speaks WebDriver protocol to the Appium server, which drives platform automation (UiAutomator2/XCUITest) on the device.

DETAILED

The test (any language with a WebDriver client) sends commands to Appium's server; based on desired capabilities, Appium delegates to the platform driver - UiAutomator2 for Android, XCUITest for iOS - which performs native interactions on the device/emulator. That architecture is why one API automates both platforms, and why capability/driver mismatches are the top setup failure.

REAL PROJECT EXAMPLE

A "session not created" error that stalled us a day was simply automationName missing - Appium tried the wrong driver for the Android version. Understanding the chain made such errors ten-minute fixes.

Q5 INTERMEDIATE**How do you make mobile automation less flaky?****QUICK ANSWER**

Stable accessibility-id locators, explicit waits for app state, controlled test data, device farm hygiene, and interruption-safe flows.

DETAILED

Ask developers for accessibility ids on interactive elements (helps users AND tests). Wait on conditions, not sleeps - mobile timing varies wildly per device. Reset app state deterministically between tests. Keep devices healthy (storage, OS popups disabled, consistent locale). And design flows to tolerate legit interruptions rather than pretending they don't happen.

REAL PROJECT EXAMPLE

Adding accessibility ids to 30 key elements cut our XPath count by 80% and halved suite flakiness in a fortnight - the single highest-ROI ask we ever made of the mobile devs.

Q6 INTERMEDIATE**What is Maestro and when would you pick it over Appium?****QUICK ANSWER**

A YAML-based mobile testing tool - faster to write and more stable for UI flows; Appium wins for complex logic and language ecosystems.

DETAILED

Maestro flows are declarative YAML with built-in tolerance for timing and minor UI shifts, near-zero setup, and easy CI use - superb for smoke/regression journey tests and for teams without deep coding capacity. Appium offers full programming power, wider platform/device cloud support, and integration into existing language frameworks - better for data-driven logic, custom helpers and complex assertions. They coexist happily.

REAL PROJECT EXAMPLE

We rebuilt our 15-flow smoke pack in Maestro in two days - it had taken two weeks in Appium and flaked weekly. Appium stayed for the data-heavy regression suite with API seeding.

Q7 ADVANCED**Design a device coverage strategy for an app with limited budget.**

QUICK ANSWER

Data-driven sampling: your analytics' top devices + oldest supported OS + one small-screen + one low-end - emulators for the rest.

DETAILED

Pull the actual user device/OS distribution from analytics and cover ~80% of sessions with 5-8 physical devices: the top sellers, the oldest OS you support, one low-RAM device, one small screen, both platforms' latest OS. Everything else runs on emulators continuously and a cloud device farm (BrowserStack/Firebase Test Lab) for release spot-checks. Revisit quarterly - the matrix drifts.

REAL PROJECT EXAMPLE

Analytics showed 6 devices covering 78% of sessions. We bought those six; a cloud farm handled long-tail spot checks per release. Escaped device-specific bugs dropped to near zero at a fraction of a lab's cost.

Q8 ADVANCED**How do you test an app's offline behaviour and data sync?**

QUICK ANSWER

State-transition testing: act offline, verify queuing/UX, reconnect, verify sync, and attack the conflict cases.

DETAILED

Map what SHOULD work offline vs degrade gracefully. Test: actions performed offline persist locally with honest UI ("will send when online"); reconnection syncs completely without duplicates; conflicts (same record edited on two devices) resolve by the documented rule; partial syncs interrupted mid-transfer recover. Airplane mode mid-request is the brutal little test that finds the crashes.

REAL PROJECT EXAMPLE

Toggling airplane mode exactly between "submit tapped" and "response received" crashed the app and - worse - double-charged on retry. That one test forced idempotency keys into the API design.

Q9 ADVANCED**How would you test push notifications end to end?**

QUICK ANSWER

Cover the chain: trigger -> delivery -> display -> tap behaviour -> permissions/settings - across app states (foreground, background, killed).

DETAILED

Verify the trigger fires (backend event), the payload is correct (title, body, deep link), delivery works per platform (FCM/APNs), display respects the app state - foreground in-app banner vs system notification - tapping deep-links to the right screen WITH the right auth state, and permission-denied/notifications-off cases degrade silently. Add localisation, Do Not Disturb, and token-refresh (reinstall) scenarios for depth.

REAL PROJECT EXAMPLE

Our order-shipped notification worked perfectly - except when tapped with the app killed, it opened the home screen instead of the order (cold-start deep link bug). Testing all three app states found what happy-path testing never would.

MODULE 8

DevOps & CI/CD

Q1 BEGINNER

Why does a tester need Git - beyond "the code lives there"?

QUICK ANSWER

Test code is code: branching, reviewing and versioning your automation, plus reading app diffs to target testing.

DETAILED

Testers use Git daily to version automation frameworks, branch per feature, open PRs for review, and bisect when suites break. Reading the application's diffs is underrated: the changed files tell you exactly where regression risk concentrates this release.

REAL PROJECT EXAMPLE

Before each release I skimmed the merged PRs; when a "small refactor" touched the pricing module, we deepened pricing regression - and caught a rounding bug the story's own tests missed.

Q2 BEGINNER

Walk me through what happens when you open a pull request.

QUICK ANSWER

Push your branch, open the PR, CI runs the checks, teammates review, you address comments, then it merges to main.

DETAILED

A PR proposes your branch's commits into main: the diff is reviewable, discussion happens inline, and required checks (build, lint, tests) must pass. Reviews catch what automation can't; automation catches what reviewers won't. Merge strategies (squash/rebase) keep history clean. For test code, PR review is where locator quality and flake-risk get challenged.

REAL PROJECT EXAMPLE

A reviewer flagged my hard-coded 5s wait in a PR - the conversation ended with a proper condition-based wait and a team lint rule banning sleeps. That's the PR process improving the suite, not just the change.

Q3 BEGINNER

What is the difference between CI and CD?

QUICK ANSWER

CI: integrate and test every change automatically. CD: keep the build deployable (delivery) or actually auto-deploy it (deployment).

DETAILED

Continuous Integration merges small changes frequently, each verified by automated build + tests. Continuous Delivery extends that: every green build CAN be released at the push of a button. Continuous Deployment goes further: green builds ship to production automatically. Testing is the enabler of all three - without trusted automation, "green" means nothing.

REAL PROJECT EXAMPLE

Our team moved from weekly release trains to on-demand deploys only after the regression suite earned

trust - the pipeline was ready long before the confidence was.

Q4 INTERMEDIATE

Design the test stages of a CI pipeline for a web app.

QUICK ANSWER

Fast -> slow: lint/unit on push; API + smoke E2E on PR; full regression nightly; deploy gates on the criticals.

DETAILED

Pyramid the pipeline: static checks and unit tests (seconds) on every push; API/integration tests and a browser smoke pack (minutes) on PRs - parallelised; the full E2E regression and non-functional suites (visual, a11y, performance) on nightly schedules; post-deploy smoke against staging/production. Each stage fails fast and reports clearly - a red pipeline nobody reads is worse than none.

REAL PROJECT EXAMPLE

We rebalanced a 50-minute PR pipeline into 12 minutes by pushing deep suites to nightly and parallelising E2E across 4 workers - PR feedback got fast enough that developers stopped bypassing it.

Q5 INTERMEDIATE

Your tests pass locally but fail in the CI container. What differs?

QUICK ANSWER

Environment: headless browser, missing deps/env vars, resources, network access, timezone/locale, and parallel data collisions.

DETAILED

CI containers are leaner and stricter: headless browsers expose timing/rendering differences; env vars and secrets may be absent; CPU/RAM limits surface races; the network may block third parties; TZ/locale defaults differ (classic date-bug source); and parallel workers collide on shared data. Reproduce locally with the same Docker image the runner uses - that usually ends the mystery.

REAL PROJECT EXAMPLE

A date-picker test failed only in CI: the container ran UTC while devs sat in IST. Pinning the app + tests to one timezone fixed it and prevented a whole class of future date flakes.

Q6 INTERMEDIATE

How would you use Docker in your testing workflow?

QUICK ANSWER

Disposable dependencies (DB, queues) per run, the app under test containerised, and consistent browser/runner images in CI.

DETAILED

Three wins: dependency containers (postgres, redis, mock services via compose) give each test run a pristine stack; running the application image from CI tests exactly what ships; and running tests in a pinned browser/runner image (e.g. Playwright's) kills "works on my machine". Add named volumes for seeded data snapshots and you get fast, reproducible integration environments.

REAL PROJECT EXAMPLE

docker compose up gave every engineer the full stack - app, DB, payment mock - in 40 seconds. Bugs stopped being "staging only" because everyone finally tested against identical environments.

Q7 ADVANCED**Flaky tests are eroding trust in your CI. What is your systematic approach?**

QUICK ANSWER

Measure and quarantine, fix the top offenders by root cause, add guardrails (no sleeps, isolated data), and defend the green.

DETAILED

First, data: track pass/fail per test to rank flakiness. Quarantine flaky tests into a non-blocking lane so main stays trustworthy while they're fixed - quarantine with an expiry, not a graveyard. Root-cause the top ten (usually: timing assumptions, shared state, order dependence, external services); fix patterns, not instances. Then prevent: lint rules, mandatory data isolation, retries treated as alarms not cures.

REAL PROJECT EXAMPLE

A flake dashboard shamed the top 10 weekly; two months of pattern-fixes took CI from 70% to 98% green, and "just re-run it" culture died - reds got investigated again because they were believable.

Q8 ADVANCED**How do you test the pipeline itself - deployments, rollbacks, migrations?**

QUICK ANSWER

Treat the pipeline as a product: rehearse deploys and rollbacks in staging, smoke post-deploy, and test migrations both ways.

DETAILED

Deployment logic deserves tests: staging deploys as full rehearsal, automated post-deploy smoke as the gate, rollback drills actually executed (a rollback never rehearsed is a rumour), DB migrations tested forward AND backward against production-shaped data, and feature flags verified in both states. Release confidence comes from rehearsed mechanics, not hope.

REAL PROJECT EXAMPLE

Our first rollback drill revealed the rollback script failed against the new DB schema - discovered in a calm staging exercise instead of a 3am incident. Migration down-scripts joined the definition of done that week.

Q9 BEGINNER**What is a merge conflict and how do you resolve one?**

QUICK ANSWER

Two branches changed the same lines; Git can't choose. You edit the conflict markers, keep the right code, test, then commit.

DETAILED

When merging, Git auto-combines non-overlapping changes; overlapping edits produce conflict markers (<<<<<< / ===== / >>>>>>) in the file. Resolution: understand BOTH changes (often talking to the other author), edit to the correct combined result, remove markers, run the tests, then stage and commit. Prevention beats cure: small branches, frequent pulls, early PRs.

REAL PROJECT EXAMPLE

My locator refactor collided with a teammate's new test using the old locators. Resolving together took five minutes and produced better code than either branch - plus a team habit of rebasing test branches daily.

MODULE 9

Advanced Testing

Q1 BEGINNER

Explain load, stress and spike testing with when you'd use each.

QUICK ANSWER

Load: expected volume vs SLAs. Stress: beyond limits to find break/recovery. Spike: sudden surges. Soak: sustained load for leaks.

DETAILED

Load testing validates SLAs at realistic concurrency before launches. Stress testing ramps until failure to learn capacity, failure mode (graceful degradation vs collapse) and recovery. Spike testing hammers instantly to model sales/viral moments. Soak/endurance runs hours to expose leaks and resource exhaustion. Each answers a different business question - name the question first.

REAL PROJECT EXAMPLE

Before a TV campaign we spike-tested at 15x normal traffic: the app held, but the email service's rate limit backed up the queue for hours - a dependency, not the app, was the bottleneck. That finding reshaped the architecture.

Q2 BEGINNER

What would you check first in a performance test report?

QUICK ANSWER

Percentiles (p95/p99) over averages, error rate under load, and throughput vs the target - averages hide the pain.

DETAILED

Averages lie: a 300ms average can hide a p99 of 8 seconds - and 1% of your users living there. I read percentile response times against SLAs, error rates as load rises (failures under load are performance bugs), throughput plateaus (the knee = capacity), and resource saturation (CPU/memory/DB connections) to locate the bottleneck.

REAL PROJECT EXAMPLE

A "passing" 400ms-average test had a p95 of 6s on checkout - connection-pool exhaustion. The average said ship it; the percentiles said fix the pool size first.

Q3 BEGINNER

What is visual regression testing and where does it fit?

QUICK ANSWER

Automated screenshot comparison against approved baselines - catching layout/style breakage functional tests can't see.

DETAILED

Tools like Percy/Applitools capture screenshots per state/viewport and diff them against baselines, flagging changes for human approve/reject. It fits AFTER functional stability (flaky functional tests make noisy baselines) and shines on style-system changes, responsive layouts and component libraries - where one CSS edit ripples everywhere.

REAL PROJECT EXAMPLE

A global CSS refactor passed all 400 functional tests; visual testing flagged 23 pages - three genuinely broken (overlapping price labels), twenty intentional. Ten minutes of review versus a broken release.

Q4 INTERMEDIATE

As a QA (not a pentester), what security testing can you meaningfully do?

QUICK ANSWER

A lot: authz matrices (IDOR), input validation, session handling, sensitive-data exposure, ZAP scans - the OWASP basics.

DETAILED

Testers own functional security: role x resource permission matrices, changing ids in URLs/APIs (IDOR), input validation (script/SQL payloads in every field), session behaviour (logout truly kills the session? tokens expire?), sensitive data in URLs/logs/responses, and password/reset flows. Add ZAP passive scanning to normal test browsing for free findings. Professional pentests complement - not replace - this continuous layer.

REAL PROJECT EXAMPLE

During routine testing I noticed the invoice URL pattern /invoices/1234.pdf was unauthenticated - incrementing the number served other companies' invoices. Highest-impact finding of my year, found with zero security tooling.

Q5 INTERMEDIATE

How would you build accessibility testing into the team's routine?

QUICK ANSWER

Automate the detectable 30-40% (axe in CI), add manual keyboard/screen-reader passes on key flows, and shift standards left into design.

DETAILED

Layer it: axe-core/Lighthouse in CI catches contrast, labels and ARIA misuse on every build; a manual keyboard-only pass and a screen-reader session per critical flow each release catches the experience automation can't judge; and WCAG criteria enter acceptance criteria so accessibility is built, not retrofitted. Assign real assistive tech (NVDA/VoiceOver) familiarity to at least one team member.

REAL PROJECT EXAMPLE

Our checkout was "compliant" per axe but unusable by keyboard - the payment iframe trapped focus. Automated zero, manual pass caught it. Both layers or you're guessing.

Q6 INTERMEDIATE

Your load test says the system handles 1,000 users, but production died at 400. What went wrong?

QUICK ANSWER

Unrealistic modelling: think time, data variety, cache hits, environment parity, or workload mix didn't match reality.

DETAILED

Classic gaps: virtual users hammering one endpoint with no think time (unrealistic but ALSO lighter than real browsing mixes); everyone using the same cached test data (100% cache hits); staging smaller than production or missing real integrations; workload mix wrong (all reads, production was write-heavy); connection reuse hiding TLS/connection costs. Model from production analytics, replay realistic journeys, and

validate the test against a known production day first.

REAL PROJECT EXAMPLE

Our test reused one product page - cache served everything. Randomising across 10k products dropped "capacity" from 1,000 to 350 users... which matched the production incident exactly. The test was finally telling the truth.

Q7 ADVANCED

How do you choose which advanced testing types a product actually needs?

QUICK ANSWER

From risk and consequence: traffic patterns -> performance; data sensitivity -> security; audience/regulation -> accessibility; UI churn -> visual.

DETAILED

Not every product needs everything. B2C spikes and revenue-per-minute demand performance engineering; regulated or PII-heavy domains mandate security depth; public-sector/consumer products carry legal accessibility obligations; design-system-driven UIs justify visual testing. I map consequences (money, law, reputation, users harmed) to investment - and defend the "no"s as explicitly as the "yes"es.

REAL PROJECT EXAMPLE

For an internal 50-user admin tool we skipped performance testing entirely but went deep on access control - wrong permissions there exposed salary data. Risk decided, not fashion.

Q8 ADVANCED

How would you introduce performance testing to a team that has none - cheaply?

QUICK ANSWER

Start with one k6 script on the top journey, thresholds in CI, production-like data volume - grow from evidence.

DETAILED

Begin small: script the single most valuable journey in k6, set explicit thresholds (p95 < 800ms, errors < 1%), run it against staging with realistic data volumes, and wire it into CI as a nightly job. Every regression it catches funds the expansion. Add scenarios, then soak tests, then production-mirroring load models as maturity grows. The trap to avoid: a giant JMeter project nobody runs.

REAL PROJECT EXAMPLE

One 80-line k6 script on checkout, run nightly, caught a 3x latency regression from an innocent-looking ORM change the same week it merged. That single catch bought performance testing its permanent CI slot.

Q9 ADVANCED

What is shift-right testing - testing in production - and how is it done safely?

QUICK ANSWER

Quality activities after release: feature flags, canary releases, monitoring, synthetic checks - controlled exposure, instant rollback.

DETAILED

Some risks only exist in production: real data shapes, real traffic, real integrations. Shift-right practices test there safely: canary deployments (release to 5% of users, watch error rates), feature flags (instant kill-switch), synthetic monitoring (scripted journeys running against prod every few minutes), and observability (alerts on

error/latency budgets). It complements - never replaces - pre-release testing.

REAL PROJECT EXAMPLE

Our checkout redesign passed every staging test, then a canary rollout showed a 2% payment-error spike for one card provider we couldn't simulate. Flag off, fix, re-canary - users barely noticed. That incident converted the whole team to shift-right.

MODULE 10

AI for Software Testing

Q1 BEGINNER

How do you use AI in your testing work today?

QUICK ANSWER

As an accelerator: draft test cases from requirements, generate test data, explain unfamiliar code, propose debug hypotheses - all human-verified.

DETAILED

My pattern: requirements -> AI drafts scenarios/cases which I prune and sharpen (it over-generates happy paths, I add domain edge cases); data generation for realistic varied records; code explanation when inheriting suites; failure triage by pasting stack traces with context. Net effect: the mechanical 60% goes faster, freeing time for the judgement 40% AI can't do.

REAL PROJECT EXAMPLE

For a 30-field insurance form, AI drafted 80 field-validation cases in minutes; I corrected the domain rules it guessed wrong (age bands, postcode logic) and added the cross-field scenarios it missed entirely. A day's work became two hours - with better coverage.

Q2 BEGINNER

What are AI's biggest limitations for testing?

QUICK ANSWER

Hallucination, no real product knowledge, generic coverage, training-data staleness - and it never says "I'm not sure".

DETAILED

Models confidently invent fields, behaviours and APIs; they don't know YOUR product's rules, history or users; generated coverage skews generic (great boundaries, weak domain logic); knowledge lags current tool versions; and confidence never signals correctness. Treat outputs as drafts from a fast, well-read intern who has never seen your app.

REAL PROJECT EXAMPLE

AI-generated API tests asserted a "status" field our API never returned - fluent, plausible, wrong. Sixty seconds with the real response caught it; blind trust would have shipped noise into CI.

Q3 BEGINNER

What makes a good prompt for testing tasks?

QUICK ANSWER

Role, full context (requirement/code/error), explicit output format, coverage expectations, and constraints - plus iteration.

DETAILED

Structure: assign a role ("senior QA engineer"); paste the actual artefact (requirement, schema, code) rather than describing it; specify output shape (table, Gherkin, code in your framework's style); name the coverage you want (boundaries, negatives, security); state constraints (tool versions, naming conventions). Then iterate - follow-ups like "now add concurrency edge cases" are where the value compounds.

REAL PROJECT EXAMPLE

The upgrade from "write tests for signup" to a structured prompt with our actual validation rules and output format took generated usefulness from ~30% keepable to ~80% - same model, better contract. The QA Prompt Generator on qatools.dev productises exactly this structure.

Q4 INTERMEDIATE

How do you use AI to speed up automation code without degrading quality?

QUICK ANSWER

Generate boilerplate (page objects, specs, helpers) in-context, then review like a junior's PR: verify selectors, assertions and conventions.

DETAILED

Copilot/Claude excel at pattern-following: new page objects mirroring existing ones, test skeletons, data builders, refactoring repetitive suites. The discipline: every suggestion gets read; selectors verified against the real DOM; assertions checked against the real contract; framework conventions enforced. AI writes faster than review reads - never let generation outpace verification.

REAL PROJECT EXAMPLE

I fed Claude one exemplar page object and the HTML of a new page; it produced a 90%-correct class in our exact style. The 10% it got wrong (a dynamic locator) is precisely why the review step is non-negotiable.

Q5 INTERMEDIATE

Where would you NOT use AI in testing?

QUICK ANSWER

Confidential data in public tools, release decisions, compliance sign-off, and anything unverifiable - judgement stays human.

DETAILED

Hard lines: secrets/PII/proprietary code into unapproved tools (policy + legal risk); autonomous quality gates (AI can't own release accountability); compliance/security attestations; and outputs I lack the expertise to verify - using AI to produce what you can't check is automation of ignorance. Also low-value: tasks where writing the prompt exceeds doing the work.

REAL PROJECT EXAMPLE

A teammate nearly pasted our production API keys inside a config file into a chatbot while debugging. We added a pre-paste sanitisation habit and an approved-tools list the same week - culture, not just rules.

Q6 INTERMEDIATE

How do you evaluate whether an AI-assisted workflow is actually helping?

QUICK ANSWER

Measure the loop, not the magic: time-to-done, defect escape rate, review burden, and how much generated output survives review.

DETAILED

I track before/after on concrete tasks: time to produce reviewed test cases; percentage of generated content kept vs rewritten (below ~50% keep-rate, the prompt or the task choice is wrong); downstream escape rates (is faster also worse?); and reviewer load. AI that shifts work from creation to heavy correction is a treadmill, not a tool.

REAL PROJECT EXAMPLE

Our AI-drafted regression cases had an 85% keep-rate for form validation but 20% for payment flows - so we kept AI on the former and hand-wrote the latter. Measurement placed the tool where it earned its keep.

Q7 ADVANCED

Design a responsible-AI-usage policy for a QA team.

QUICK ANSWER

Approved tools + data rules, verification-before-use as doctrine, attribution in PRs, measured rollout, and shared prompt patterns.

DETAILED

Pillars: tooling (approved list, enterprise accounts with no-training guarantees); data (no secrets/PII/client code in public tools - with sanitisation examples); verification (generated artefacts reviewed like human work, tagged in PRs so reviewers know); competence (prompt-pattern library, pairing sessions); measurement (keep-rates, escape rates) to steer investment. The goal is leverage with accountability - the human always owns the output.

REAL PROJECT EXAMPLE

We introduced a "AI-assisted" PR label. Counter-intuitively, review quality ROSE on labelled PRs - reviewers checked assertions harder. The label made verification cultural instead of aspirational.

Q8 ADVANCED

How do you see AI changing the QA role - and how do you stay valuable?

QUICK ANSWER

Execution commoditises; judgement appreciates. Move up the stack: risk analysis, strategy, AI-output verification, and testing AI systems themselves.

DETAILED

Drafting cases, boilerplate automation and triage summaries are commoditising fast. What appreciates: deciding what to test and why (risk), designing quality strategies, verifying AI outputs (a new core skill), domain depth AI lacks, and testing AI-powered features themselves - nondeterminism, bias, evaluation datasets - which classical techniques don't cover. The testers at risk are those whose whole value was typing what others decided.

REAL PROJECT EXAMPLE

Testing our first LLM-powered feature flipped my toolkit: no fixed expected outputs, so we built evaluation rubrics, golden datasets and drift monitoring. That project taught me more about my future value than any prompt trick - someone must test the AI, with real testing discipline.

Q9 BEGINNER

Will AI replace software testers?

QUICK ANSWER

It replaces tasks, not the role: execution and drafting commoditise, while judgement, risk analysis and verifying AI itself grow in value.

DETAILED

AI already writes draft cases, generates data and triages failures - so testers whose whole value was typing test cases are genuinely exposed. But deciding WHAT to test, owning quality risk, exploratory skill, domain knowledge, and verifying AI outputs (including testing AI-powered features, which behave

nondeterministically) are growing needs. The honest answer: AI replaces testers who refuse to use AI - with testers who do.

REAL PROJECT EXAMPLE

In my team AI cut test-case drafting time by more than half; nobody lost their job - the time moved into exploratory testing and building an evaluation suite for our own AI feature, work that didn't exist two years ago.

MODULE 11

Career Preparation

Q1 BEGINNER

Tell me about yourself. (Practise YOUR answer.)

QUICK ANSWER

Who you are -> years/domains -> core skills -> one recent achievement -> why this role. Under 90 seconds.

DETAILED

Structure beats improvisation: identity ("QA engineer with 3 years across e-commerce and fintech"), craft ("manual + automation with Playwright and API testing"), proof ("recently built a CI smoke suite that cut release checks from a day to 20 minutes"), motivation ("excited by this role's automation-first approach"). Tailor the skills and proof per company; keep energy in your voice.

REAL PROJECT EXAMPLE

Record yourself once - everyone is shocked the first time. Cut every "um", every irrelevant detail, and everything past 90 seconds. Then re-record until you'd hire you.

Q2 BEGINNER

Why should we hire you as a fresher with no commercial experience?

QUICK ANSWER

Evidence over experience: real projects, a public portfolio, fundamentals you can demonstrate live, and visible learning velocity.

DETAILED

Replace experience with proof-of-work: a GitHub automation framework with CI, documented bug reports from exploratory sessions on public sites, an ISTQB or solid fundamentals you can defend under questioning, and a learning trajectory (this roadmap!) showing you self-drive. Frame it: "I've done the job's activities, just not on payroll yet - here's the evidence."

REAL PROJECT EXAMPLE

A mentee with zero experience walked interviews through her Playwright repo: real POM, real CI runs, a README explaining her choices. She out-interviewed candidates with two years' experience because her evidence was inspectable.

Q3 BEGINNER

Where do you see yourself in 3 years, and how does QA fit that?

QUICK ANSWER

Show intent: deepening automation/SDET skills, owning quality processes, mentoring - a trajectory, not a placeholder job.

DETAILED

Interviewers probe commitment: is QA your path or your fallback? A strong answer names a direction (SDET, test lead, quality coach), the skills you're building toward it, and why the company's context helps. Avoid both "I haven't thought about it" and "I want to move to development ASAP" - the second tells them you'll leave the role they're hiring for.

REAL PROJECT EXAMPLE

"In three years I want to be the engineer who designed this team's automation strategy - I'm building toward that with framework design and CI depth now" landed the offer; it made the hire feel like an investment.

Q4 INTERMEDIATE

Walk me through the automation framework on your GitHub.

QUICK ANSWER

Architecture -> why those choices -> a test's journey -> CI -> what you'd improve. Own every line.

DETAILED

Structure the tour: stack and why (Playwright/TS for auto-wait and traces); layout (pages/, tests/, fixtures/, data/); how a test flows through fixtures and page objects; CI triggers and reporting; then self-awareness - what you'd refactor next. Rehearse this: "walk me through your project" is the single most predictable technical question for portfolio-carrying candidates, and hesitation on your own code is fatal.

REAL PROJECT EXAMPLE

Practise aloud with the repo open, five times. One candidate told me the "what would you improve" part - retiring their older keyword-driven layer - carried the whole interview: it proved judgement, not just construction.

Q5 INTERMEDIATE

Describe a difficult bug you found. (Have this story polished.)

QUICK ANSWER

A STAR story showing investigation skill: weird symptom -> systematic isolation -> root cause -> impact prevented.

DETAILED

Choose a bug that showcases method, not luck: intermittent failure you cornered via logs, an integration mismatch you proved with data, a race condition you reproduced deliberately. Tell it as: context, what made it hard, YOUR investigative steps, the root cause, and the quantified save. This one story can carry severity/priority, debugging and communication questions all at once.

REAL PROJECT EXAMPLE

My best: a payment double-charge appearing only on slow connections - network throttling + request logs proved a missing idempotency key. "Found it, proved it, prevented ~£X in refunds" is a complete interview arc in ninety seconds.

Q6 INTERMEDIATE

How do you tailor applications instead of mass-applying?

QUICK ANSWER

Mirror each JD: reorder skills, echo exact keywords, tune two resume bullets and the headline per role family.

DETAILED

Maintain a master resume, then per application: pull the JD's repeated terms (they're the ATS filter AND the hiring manager's priorities), mirror exact spellings, reorder your skills section to lead with their stack, and adapt your top two bullets toward their domain. Fifteen minutes per application, dramatically better response rates than a hundred identical PDFs.

REAL PROJECT EXAMPLE

Same person, same experience: 40 generic applications -> 2 responses; 15 tailored ones -> 6 interviews. The

qatools.dev/ai-cv-review tool's missing-keywords check makes the mirroring step mechanical.

Q7 ADVANCED

In a system-design-ish QA round: "How would you set up quality for a new product from day one?"

QUICK ANSWER

A strategy answer: risk profile -> test pyramid -> automation-first CI -> data/environments -> metrics - with trade-offs stated.

DETAILED

Show senior thinking: understand the product's risk profile first (money? PII? scale?); propose layered coverage (unit by devs, API/integration as the workhorse, thin E2E); CI gates from week one; test data and environment strategy early (the usual bottleneck); quality metrics that steer (escape rate, cycle time) not vanity counts. Name trade-offs explicitly - what you'd defer and why. Interviewers reward the reasoning more than the checklist.

REAL PROJECT EXAMPLE

I answered this live by sketching the pyramid and saying "I'd skip UI automation entirely for the first month - the UI is churning; API contracts are where confidence is cheap." Arguing a deliberate NO impressed more than any tool list.

Q8 ADVANCED

You have three offers. How do you evaluate them beyond salary?

QUICK ANSWER

Growth trajectory: mentorship, modern practices, automation maturity, domain depth, and who you'll learn from daily.

DETAILED

Early-career compounding beats initial salary: Will seniors review your work? Is automation real or aspirational there ("manual now, automation soon" often means never)? Modern stack and CI culture? Domain that deepens your story (fintech/health = transferable rigour)? Interview signals: how they treated you, the questions THEY asked, whether testers attend design discussions. The right first three years multiply every later offer.

REAL PROJECT EXAMPLE

I took £3k less for a team with strong code review and real CI. Two years later that decision was worth several times the gap in both salary and options - the compounding was the salary.

Q9 INTERMEDIATE

What questions should YOU ask at the end of an interview?

QUICK ANSWER

Ask about quality culture, automation reality, and success criteria - signal seniority and gather decision data at the same time.

DETAILED

Strong closers: "How does testing fit in your development process - when do testers get involved?", "What does your automation actually cover today?" (reveals aspiration vs reality), "How do you handle a release with known defects?" (risk culture), "What would a great first 90 days look like for this role?". Avoid

salary/leave in round one, and never say "no questions" - it reads as no interest.

REAL PROJECT EXAMPLE

Asking "what happens when QA says no-go but the deadline says ship?" produced a revealingly awkward silence at one company and a brilliant answer at another. I chose the second - and the question itself was cited in my offer feedback.